

Concept MAP : MODEL-CHECKING VERIFICATION PROCESSING

Design Specification
Architecture
RTL/code
Environment
Assumptions

1. SYSTEM MODEL (Design)
→ Finite state model
→ eg, FSM, Kripke, Structure, Transition System

2. Property Specification (Temporal Logic)
• LTL (Linear Temporal Logic)
• CTL (Computation Tree Logic)
• Express correctness requirements
• Safety, Liveness, Fairness etc

Examples (Safety)
• $\neg \text{error}$
• $\neg (\text{req} \rightarrow F)$
Examples (Liveness)
• $F \text{ done}$
• $G(\text{req} \rightarrow F \text{ done})$

3. VERIFICATION ENGINE

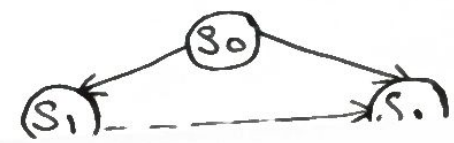
- Takes System Model and Property
- Builds state space
- Checks if Property holds

5. STATE SPACE EXPLOSION

- Number of states grows exponentially
- Leads to huge memory and time

4. STATE SPACE EXPLORATION

- Generate reachable states
- Explores transitions
- Uses algorithms



6. ABSTRACTION TECHNIQUES

→ Reduce state space while preserving properties

→ Techniques:

- Predicate Abstraction
- Symmetry Reduction

Property Holds?

YES

Verification Success
System satisfies the property

7. Counterexample Generation

- Provides execution trace that violates the properties



(err S1)

8. DESIGN REFINEMENT

- Analyze counterexample
- Fix design / Strengthen property
- Update model
- Re-verify

SUMMARY

Model checking systematically explores all possible behaviors of a system model and verifies temporal logic properties. Abstractions help manage state explosion. Counterexamples guide design refinement until the system is proven correct.